逆向TV-1

逆向TV之小白蠕动学习逆向

平台为NSSCTF，标题前为id

彩笔。。

.2956[HNCTF 2022 WEEK2]e@sy_flower

题目描述：

我的IDA坏了???

题目提示为花指令

花指令即为了迷惑分析而在正常的代码中插入垃圾代码（不承担真实业务逻辑）干扰静态分析、甚至起到破坏反编汇的作用

https://blog.csdn.net/Sciurdae/article/details/133750478 这篇大佬的文章有花指令的介绍和处理方法

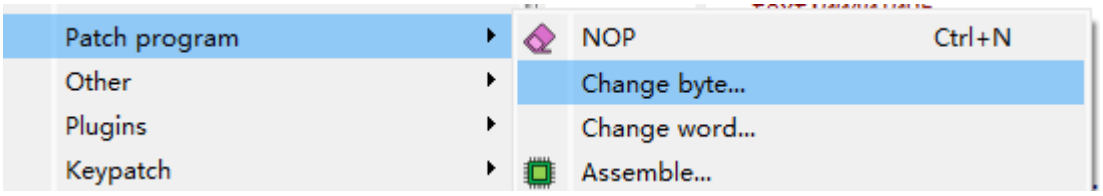
分析程序，在反汇编窗口找到爆红语句

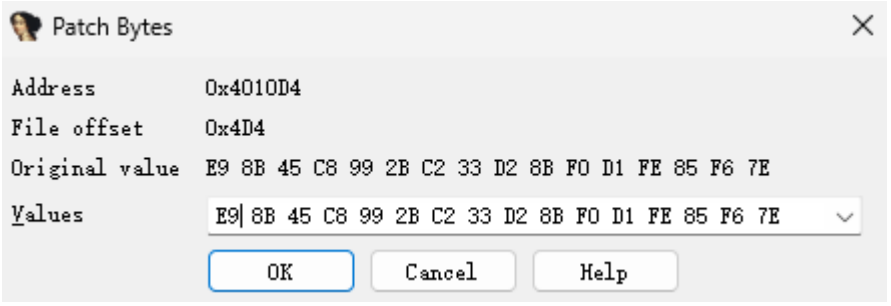
```
.text:004010D0                 jz      short near ptr loc_4010D4+1
.text:004010D2                 jnz     short near ptr loc_4010D4+1
.text:004010D4
.text:004010D4 loc_4010D4:                             ; CODE XREF: .text:004010D0↑j
.text:004010D4                                         ; .text:004010D2↑j
.text:004010D4                 jmp     near ptr 9A085664h
```

上面有jz和jnz这一对互补跳转语句，结尾的地址都+1了

jz和jnz互补语句，不论条件是否成立程序都会从地址+1处开始执行，然而ida静态分析字节会从原地址开始分析，以致代码字节分割错位，造成反编译混乱

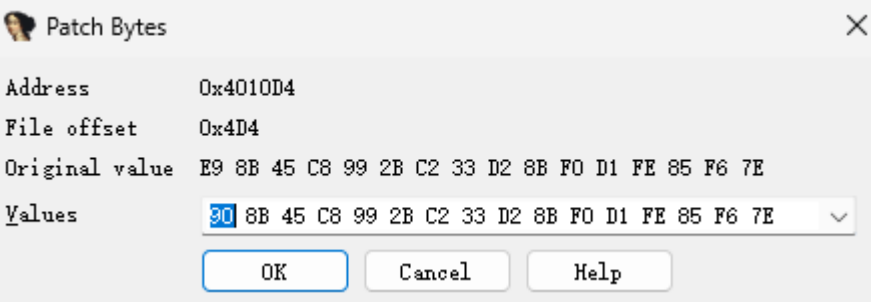
选中地址，Edit -> Patch program -> Change byte直接改字节码





既然它是地址+1，我们就把第一个字节给nop掉，让字节分割顺序恢复正常

（90一般是机器码的nop字节）



恢复正常

```
.text:004010CD                  mov     [ebp-38h], eax
.text:004010D0                  jz      short loc_4010D5
.text:004010D2                  jnz     short loc_4010D5
.text:004010D4                  nop
.text:004010D5
.text:004010D5 loc_4010D5:                              ; CODE XREF: .text:004010D0↑j
.text:004010D5                                          ; .text:004010D2↑j
.text:004010D5                  mov     eax, [ebp-38h]
```

找到main函数，确定入口按p重建函数

```
.text:00401090 ; int __cdecl main(int argc, const char **argv, const char **envp)
.text:00401090 _main            proc near               ; CODE XREF: __scrt_common_main_seh(void)+F5↓p
.text:00401090
.text:00401090 var_38           = dword ptr -38h
.text:00401090 var_34           = byte ptr -34h
.text:00401090 var_33           = byte ptr -33h
```

都恢复了，直接看伪码

```
1  int __cdecl __noreturn main(int argc, const char **argv, const char **envp)
2  {
3    signed int v3; // kr00_4
4    int i; // edx
5    char v5; // cl
6    unsigned int j; // edx
7    int v7; // eax
8    char v8[48]; // [esp+10h] [ebp-34h] BYREF
9
10   sub_401020("please input flag\n");
11   sub_401050("%s", v8);
12   v3 = strlen(v8);
13   for ( i = 0; i < v3 / 2; ++i )
14   {
15     v5 = v8[2 * i];
16     v8[2 * i] = v8[2 * i + 1];
17     v8[2 * i + 1] = v5;
18   }
19   for ( j = 0; j < strlen(v8); ++j )
20     v8[j] ^= 0x30u;
21   v7 = strcmp(v8, "c~scvdzKCEoDEZ[^roDICUMC");
22   if ( v7 )
23     v7 = v7 < 0 ? -1 : 1;
24   if ( !v7 )
25   {
26     sub_401020("yes");
27     exit(0);
28   }
29   sub_401020("error");
30   exit(0);
31 }
```

主要逻辑是这一段

```
for ( i = 0; i < v3 / 2; ++i )
{
  v5 = v8[2 * i];
  v8[2 * i] = v8[2 * i + 1];
  v8[2 * i + 1] = v5;
}
for ( j = 0; j < strlen(v8); ++j )
  v8[j] ^= 0x30u;
v7 = strcmp(v8, "c~scvdzKCEoDEZ[^roDICUMC");
```

反写脚本

```
int main()
{
char v8[] = "c~scvdzKCEoDEZ[^roDICUMC";
signed int v3 = strlen(v8);
char v5;
    for (int j = 0; j < strlen(v8); ++j )
   {
    v8[j] ^= 0x30u;
   }
    for (int i = 0; i < v3 / 2; ++i )
  {
    v5 = v8[2 * i];
    v8[2 * i] = v8[2 * i + 1];
    v8[2 * i + 1] = v5;
  }
  printf("%s",v8);
}
```

```cpp
#include<bits/stdc++.h>

int main()
{
char v8[] = "c~scvdzKCEoDEZ[^roDICUMC";
signed int v3 = strlen(v8);
char v5;
    for (int j = 0; j < strlen(v8); ++j )
   {
    v8[j] ^= 0x30u;
   }
    for (int i = 0; i < v3 / 2; ++i )
  {
    v5 = v8[2 * i];
    v8[2 * i] = v8[2 * i + 1];
    v8[2 * i + 1] = v5;
  }
  printf("%s",v8);
}
```

```
NSSCTF{Just_junk_Bytess}
--------------------------------
Process exited after 0.09008 seconds with return valu
请按任意键继续. . .
```

NSSCTF{Just_junk_Bytess}

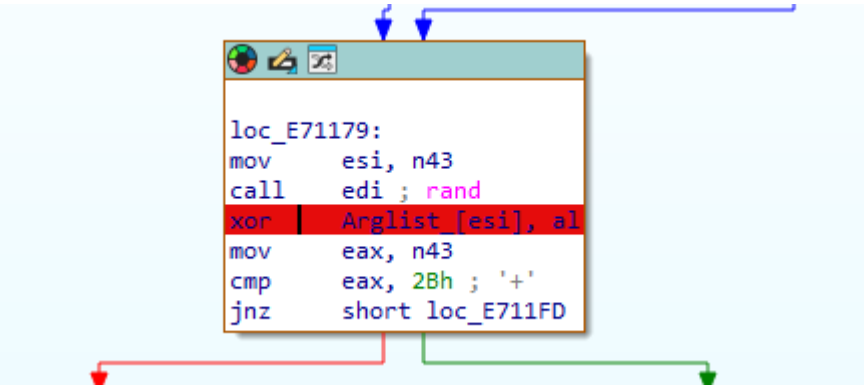
.1127[MTCTF 2021]Random

ida看伪码

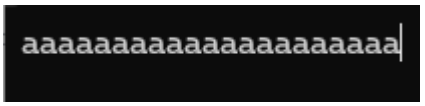
```
4  unsigned int Seed; // eax
5  int n43; // esi
6  int n42; // edx
7  int n42_1; // ecx
8  int v8; // esi
9  int n42_2; // eax
10 char *p_congratulation__n; // eax
11 int ArgList; // [esp-4h] [ebp-2Ch]
12 const char **argv_1; // [esp+0h] [ebp-28h]
13 const char **envp_1; // [esp+4h] [ebp-24h]
14
15 if ( !::n43 )
16   sub_4010B0(v3, "%s", (char)Arglist_);
17 Seed = rand();
18 srand(Seed);
19 n43 = ::n43;
20 Arglist_[n43] ^= rand();
21 if ( ::n43 == 43 )
22 {
23   ::n43 = 0;
24   n42 = 0;
25   n42_1 = 0;
26   v8 = 0;
27   n42_2 = 0;
28   do
29   {
30     if ( Arglist_[n42_2] != byte_402138[v8] )
31       break;
32     v8 = ++n42;
33     n42_1 = n42;
34     ::n43 = n42;
35     n42_2 = n42;
36   }
37   while ( n42 < 42 );
38   p_congratulation__n = "fake input..\n";
39   if ( n42_1 == 42 )
40     p_congratulation__n = "congratulation!\n";
41   sub_401050(n42_1, p_congratulation__n, ArgList);
42   return 0;
43 }
44 else
45 {
46   ++::n43;
47   main(ArgList, argv_1, envp_1);
48   return 0;
49 }
50 }
```

在异或随机数，建议动调

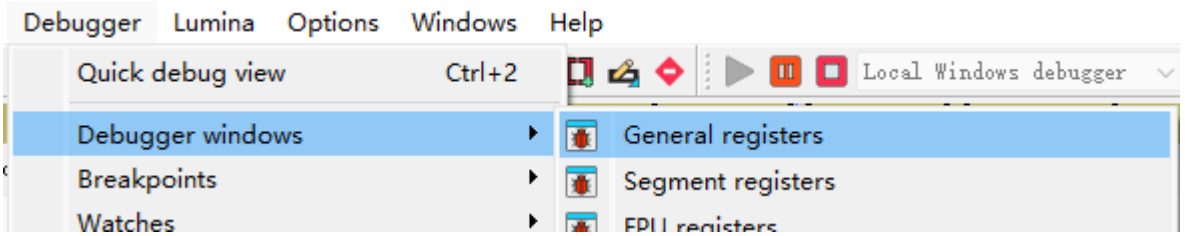
找点核心逻辑（异或）处下F2下断点，F9开始运行



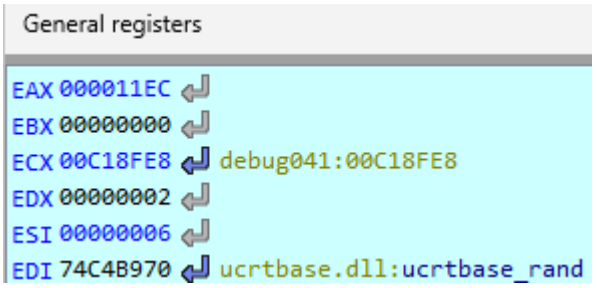
输入任意字符串

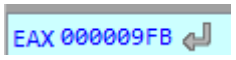


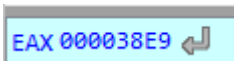
Debugger -> Debugger windows -> General registers打开寄存器窗口，EAX会记录每一次的随机数值

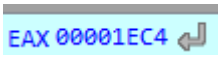


运行到断点处，每一次EAX都寄存了不同的随机数值







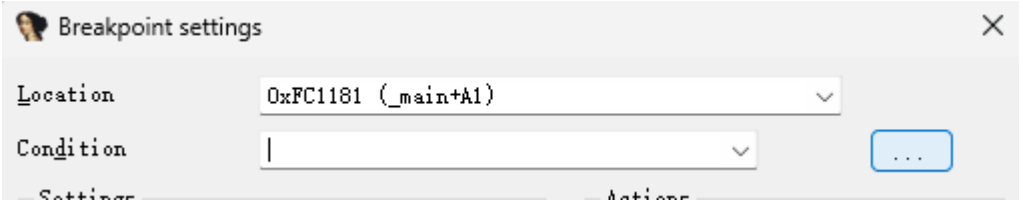


不嫌麻烦的话可以手记

但我们可以右键断点处选择edit breakpoint

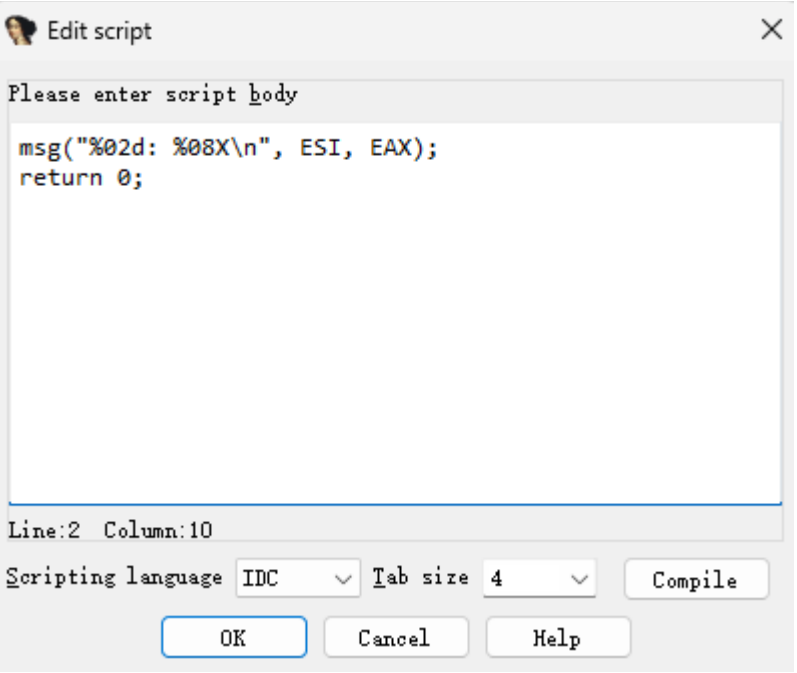


点三个点



粘贴c语言代码

```
msg("%02d: %08X\n", ESI, EAX);
return 0;
```



这样就可以把 ESI 和 EAX 按格式打印到 Output 窗口

```
00: 00000258
FC1146: Integer divide by zero (exc.code c0000094, tid 34
01: 000039A1
FC1146: Integer divide by zero (exc.code c0000094, tid 34
```

不断运行直至结束

```
00: 00000258
FC1146: Integer divide by zero (exc.code c0000094, tid 3476)
01: 000039A1
FC1146: Integer divide by zero (exc.code c0000094, tid 3476)
02: 000074CB
FC1146: Integer divide by zero (exc.code c0000094, tid 3476)
[autohidden] FC1146: Integer divide by zero (exc.code c0000094, tid 3476) -> OK
03: 000049E9
FC1146: Integer divide by zero (exc.code c0000094, tid 3476)
[autohidden] FC1146: Integer divide by zero (exc.code c0000094, tid 3476) -> OK
04: 000049ED
FC1146: Integer divide by zero (exc.code c0000094, tid 3476)
[autohidden] FC1146: Integer divide by zero (exc.code c0000094, tid 3476) -> OK
05: 0000682C
FC1146: Integer divide by zero (exc.code c0000094, tid 3476)
[autohidden] FC1146: Integer divide by zero (exc.code c0000094, tid 3476) -> OK
06: 000011EC
FC1146: Integer divide by zero (exc.code c0000094, tid 3476)
[autohidden] FC1146: Integer divide by zero (exc.code c0000094, tid 3476) -> OK
07: 000009FB
FC1146: Integer divide by zero (exc.code c0000094, tid 3476)
[autohidden] FC1146: Integer divide by zero (exc.code c0000094, tid 3476) -> OK
08: 000038E9
FC1146: Integer divide by zero (exc.code c0000094, tid 3476)
[autohidden] FC1146: Integer divide by zero (exc.code c0000094, tid 3476) -> OK
09: 00001EC4
FC1146: Integer divide by zero (exc.code c0000094, tid 3476)
[autohidden] FC1146: Integer divide by zero (exc.code c0000094, tid 3476) -> OK
10: 00001116
FC1146: Integer divide by zero (exc.code c0000094, tid 3476)
[autohidden] FC1146: Integer divide by zero (exc.code c0000094, tid 3476) -> OK
11: 00004797
FC1146: Integer divide by zero (exc.code c0000094, tid 3476)
[autohidden] FC1146: Integer divide by zero (exc.code c0000094, tid 3476) -> OK
12: 00004399
FC1146: Integer divide by zero (exc.code c0000094, tid 3476)
[autohidden] FC1146: Integer divide by zero (exc.code c0000094, tid 3476) -> OK
13: 000076B1
FC1146: Integer divide by zero (exc.code c0000094, tid 3476)
[autohidden] FC1146: Integer divide by zero (exc.code c0000094, tid 3476) -> OK
```

由前面的伪码可知有42个随机数

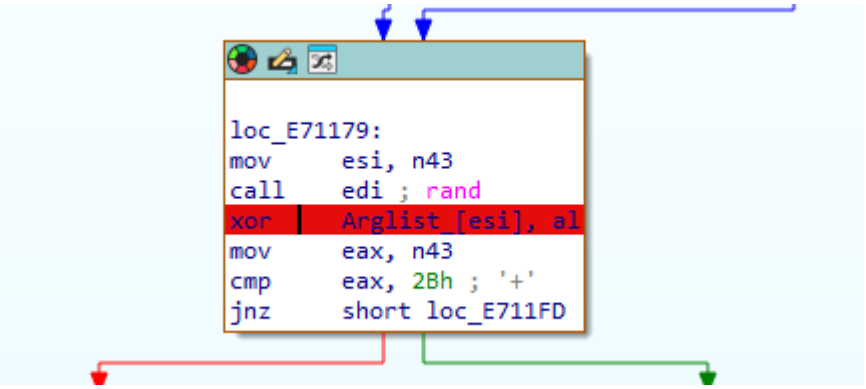
```
while ( n43_1 < 42 );
p_congratulation__n = "fake input..\n";
if ( n42 == 42 )
```

去除噪声，保留0~41位

> 00: 00000258
> 01: 000039A1
> 02: 000074CB
> 03: 000049E9
> 04: 000049ED
> 05: 0000682C
> 06: 000011EC
> 07: 000009FB
> 08: 000038E9
> 09: 00001EC4
> 10: 00001116
> 11: 00004797

> 12: 00004399
> 13: 000076B1
> 14: 000070A4
> 15: 000003E9
> 16: 000000C3
> 17: 000054C6
> 18: 00000B80
> 19: 000014BF
> 20: 0000073E
> 21: 00004F44
> 22: 00002D18
> 23: 00005C2E
> 24: 00006B73
> 25: 00000956
> 26: 00003752
> 27: 00000BB8
> 28: 0000635B
> 29: 00002066
> 30: 00007DED
> 31: 00005BBC
> 32: 00006C8A
> 33: 000067D8
> 34: 00005236
> 35: 0000478F
> 36: 000059E6
> 37: 00004ED3
> 38: 00002EB1
> 39: 00005F51
> 40: 000057B9
> 41: 00000159

由前面可知用的是al，只用aex的低8位，即最后两个字母



整理

> 58 A1 CB E9 ED 2C EC FB E9 C4 16 97 99 B1 A4 E9
> C3 C6 80 BF 3E 44 18 2E 73 56 52 B8 5B 66 ED BC
> 8A D8 36 8F E6 D3 B1 51 B9 59

密钥拿到了再与原密文逐项异或

```
{
  if ( Arglist_[n43_2] != byte_FC2138[v8] )
    break;
.rdata:00FC2138 byte_FC2138     db 3Eh                  ; DATA XREF: _main+D6↑r
.rdata:00FC2139                 db 0CDh
.rdata:00FC213A                 db 0AAh
.rdata:00FC213B                 db  8Eh
.rdata:00FC213C                 db  96h
.rdata:00FC213D                 db  1Fh
.rdata:00FC213E                 db  89h
.rdata:00FC213F                 db 0CDh
.rdata:00FC2140                 db 0DBh
.rdata:00FC2141                 db 0F1h
.rdata:00FC2142                 db  70h ; p
.rdata:00FC2143                 db 0F2h
.rdata:00FC2144                 db 0A9h
.rdata:00FC2145                 db  9Ch
.rdata:00FC2146                 db 0C2h
.rdata:00FC2147                 db  8Bh
.rdata:00FC2148                 db 0F2h
.rdata:00FC2149                 db 0FEh
.rdata:00FC214A                 db 0ADh
.rdata:00FC214B                 db  8Bh
.rdata:00FC214C                 db  58h ; X
.rdata:00FC214D                 db  7Ch ; |
.rdata:00FC214E                 db  2Fh ; /
.rdata:00FC214F                 db    3
.rdata:00FC2150                 db  4Ah ; J
.rdata:00FC2151                 db  65h ; e
.rdata:00FC2152                 db  31h ; 1
.rdata:00FC2153                 db  89h
.rdata:00FC2154                 db  76h ; v
.rdata:00FC2155                 db  57h ; W
.rdata:00FC2156                 db  88h
.rdata:00FC2157                 db 0DFh
.rdata:00FC2158                 db 0B8h
.rdata:00FC2159                 db 0E9h
.rdata:00FC215A                 db    1
.rdata:00FC215B                 db 0E9h
.rdata:00FC215C                 db 0DEh
.rdata:00FC215D                 db 0E5h
.rdata:00FC215E                 db  86h
.rdata:00FC215F                 db  68h ; h
.rdata:00FC2160                 db  8Fh
.rdata:00FC2161                 db  24h ; $
.rdata:00FC2162                 db 0D3h
.rdata:00FC2163                 db  5Ah ; Z
```

脚本

```
int main() {
     char a[] = {
        0x3E, 0xCD, 0xAA, 0x8E, 0x96, 0x1F, 0x89, 0xCD,
        0xDB, 0xF1, 0x70, 0xF2, 0xA9, 0x9C, 0xC2, 0x8B,
        0xF2, 0xFE, 0xAD, 0x8B, 0x58, 0x7C, 0x2F, 0x03,
        0x4A, 0x65, 0x31, 0x89, 0x76, 0x57, 0x88, 0xDF,
        0xB8, 0xE9, 0x01, 0xE9, 0xDE, 0xE5, 0x86, 0x68,
        0x8F, 0x24
    };
     char b[] = {
        0x58, 0xA1, 0xCB, 0xE9, 0xED, 0x2C, 0xEC, 0xFB,
        0xE9, 0xC4, 0x16, 0x97, 0x99, 0xB1, 0xA4, 0xE9,
        0xC3, 0xC6, 0x80, 0xBF, 0x3E, 0x44, 0x18, 0x2E,
```

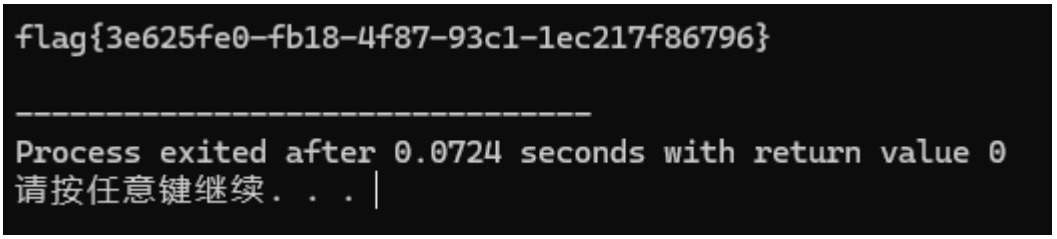
```
        0x73, 0x56, 0x52, 0xB8, 0x5B, 0x66, 0xED, 0xBC,
        0x8A, 0xD8, 0x36, 0x8F, 0xE6, 0xD3, 0xB1, 0x51,
        0xB9, 0x59
    };
    char c[43];

    for (int i = 0; i < 42; i++) {
        c[i] = a[i] ^ b[i];
    }

    c[42] = '\0';

    printf("%s\n", c);
    return 0;
}
```

flag{3e625fe0-fb18-4f87-93c1-1ec217f86796}



.6522[GHCTF 2025]Canon

题目描述：Kanon und Gigue in D-Dur für drei Violinen und Basso Continuo

依旧ida看伪码

```
sub_140002470("Enter the flag: ", argv, envp);
  sub_1400024F0("%36s", Str);
  if ( strlen(Str) == 36 )
  {
    strncpy(Destination, Str, 0xCu);
    n12 = 12;
    Destination[12] = 0;
    strncpy(Str1, Source, 0xCu);
    n12_1 = 12;
    Str1[12] = 0;
    strncpy(Destination_, Source_, 0xCu);
    n12_2 = 12;
    Destination_[12] = 0;
    v15 = 0x500000001LL;
    n6 = 6;
    n3 = 3;
    n4 = 4;
    v19 = 1;
    n4_1 = 4;
    n5 = 5;
    v22[0] = 0;
    v22[1] = 1;
```

```
    v22[2] = 2;
    v11[0] = 0;
    v11[1] = 0;
    v11[2] = 0;
    for ( n8_1 = 0; n8_1 < 8; ++n8_1 )
    {
      for ( n2 = 0; n2 < 3; ++n2 )
      {
        if ( n8_1 >= v22[n2] )
        {
          n8 = v11[n2];
          if ( n8 < 8 )
          {
            if ( n2 )
            {
              if ( n2 == 1 )
              {
                sub_1400015D0(Str1, Destination_, *((unsigned int *)&v15 + n8));
              }
              else if ( n2 == 2 )
              {
                sub_1400015D0(Destination_, Destination, *((unsigned int *)&v15 + n8));
              }
            }
            else
            {
              sub_1400015D0(Destination, Str1, *((unsigned int *)&v15 + n8));
            }
            ++v11[n2];
          }
        }
      }
    }
    if ( !strcmp(Destination, "WgvDmssEvcY326bHo3nNro3vXvvfmgrz")
      && !strcmp(Str1, "gX+Ri9PG=bt5=OOB6hscPQOL")
      && !strcmp(Destination_, "T6bHgUPL2gXUd=xT=FNHtPzV") )
    {
      sub_140002470("Congratulations! You have found the flag!\n", argv_1, envp_1);
    }
    else
    {
      sub_140002470("Invalid flag!\n", argv_1, envp_1);
    }
    return 0;
  }
  else
  {
    sub_140002470("Invalid flag!\n", argv_2, envp_2);
    return 0;
  }
}
```

一步一步来

```
sub_1400024F0("%36s", Str);
if ( strlen(Str) == 36 )
```

确认Str为36位长

```
strncpy(Destination, Str, 0xCu);
n12 = 12;
Destination[12] = 0;
strncpy(Str1, Source, 0xCu);
n12_1 = 12;
Str1[12] = 0;
strncpy(Destination_, Source_, 0xCu);
n12_2 = 12;
Destination_[12] = 0;
```

将Str分成Destination，Str1，Destination_三份，每份12位长

```
for ( n8_1 = 0; n8_1 < 8; ++n8_1 )
{
  for ( n2 = 0; n2 < 3; ++n2 )
  {
    if ( n8_1 >= v22[n2] )
    {
      n8 = v11[n2];
      if ( n8 < 8 )
      {
        if ( n2 )
        {
          if ( n2 == 1 )
          {
            sub_1400015D0(Str1, Destination_, *((unsigned int *)&v15 + n8));
          }
          else if ( n2 == 2 )
          {
            sub_1400015D0(Destination_, Destination, *((unsigned int *)&v15 + n8));
          }
        }
        else
        {
          sub_1400015D0(Destination, Str1, *((unsigned int *)&v15 + n8));
        }
        ++v11[n2];
      }
    }
  }
}
```

结合前面对v22和v11的定义，这一段是用函数sub_1400015D0处理字符串，再取一数，循环8次

*((unsigned int *)&v15 + n8结合前面的定义

```
v15 = 0x500000001LL;
n6 = 6;
n3 = 3;
n4 = 4;
v19 = 1;
n4_1 = 4;
n5 = 5;
```

v15 = 0x500000001LL 64位数，被int定义从8字节拆为4字节，分为5和1，小端存储按顺序就是1和5

相当于数组{1,5,6,3,4,1,4,5}8个数刚好对应前面的8次循环

即：

n2==0时，Destination为字符串，Str1为key，数组内取一数

n2==1时，Str1为字符串，Destination_为key，数组内取一数

n2==2时，Destination_为字符串，Destination为key，数组内取一数

```
if ( !strcmp(Destination, "WgvDmssEvcY326bHo3nNro3vXvvfmgrz")
  && !strcmp(Str1, "gX+Ri9PG=bt5=00B6hscPQOL")
  && !strcmp(Destination_, "T6bHgUPL2gXUd=xT=FNHtPzV") )
```

最后把校验的字符串给出来了

我们对函数sub_1400015D0进行分析

一个switch语句，给出了1~7的情况，前面的数组内没有2和7，不用管，分析1,3,4,5,6

```
v9 = strlen(p_Str1);
  v21 = strlen(Destination);
  switch ( a3 )
  {
    case 1:
      for ( i = 0; i < v9; ++i )
      {
        v22 = Destination[i % v21];
        if ( p_Str1[i] < 65 || p_Str1[i] > 90 )
        {
          if ( p_Str1[i] < 97 || p_Str1[i] > 122 )
          {
            if ( p_Str1[i] >= 48 && p_Str1[i] <= 57 )
              p_Str1[i] = (p_Str1[i] + v22 - 48) % 10 + 48;
          }
          else
          {
            p_Str1[i] = (p_Str1[i] + v22 - 97) % 26 + 97;
          }
        }
        else
        {
          p_Str1[i] = (p_Str1[i] + v22 - 65) % 26 + 65;
        }
      }
      break;
    case 2:
      v32[0] = 1;
      v32[1] = 3;
      v32[2] = 5;
      v32[3] = 7;
      v32[4] = 9;
      v32[5] = 11;
      v32[6] = 15;
      v32[7] = 17;
      v32[8] = 19;
```

```
    v32[9] = 21;
    v32[10] = 23;
    v32[11] = 25;
    for ( j = 0; j < v9; ++j )
    {
      if ( p_Str1[j] < 65 || p_Str1[j] > 90 )
      {
        if ( p_Str1[j] >= 97 && p_Str1[j] <= 122 )
          p_Str1[j] = (Destination[j % v21] + v32[j % 12] * (p_Str1[j] - 97)) % 26 + 97;
      }
      else
      {
        p_Str1[j] = (Destination[j % v21] + v32[j % 12] * (p_Str1[j] - 65)) % 26 + 65;
      }
    }
    break;
  case 3:
    v10 = *Destination % 10 + 2;
    v33 = v10;
    Block = malloc(saturated_mul(v10, 8u));
    for ( k = 0; k < v10; ++k )
    {
      Block[k] = malloc(v9 + 1);
      memset((void *)Block[k], 0, v9 + 1);
    }
    for ( m = 0; v10 * m < v9; ++m )
    {
      for ( n = 0; n < v10 && n + v10 * m < v9; ++n )
        *(_BYTE *)(Block[n] + m) = p_Str1[n + v10 * m];
    }
    v18 = 0;
    for ( ii = 0; ii < v10; ++ii )
    {
      for ( jj = 0; jj < m; ++jj )
      {
        if ( *(_BYTE *)(Block[ii] + jj) && v18 < v9 )
          p_Str1[v18++] = *(_BYTE *)(Block[ii] + jj);
      }
    }
    p_Str1[v18] = 0;
    for ( kk = 0; kk < v10; ++kk )
      free((void *)Block[kk]);
    free(Block);
    break;
  case 4:
    v25 = *Destination % 10 + 2;
    for ( mm = 0; mm < v25; ++mm )
    {
      v11 = p_Str1[v9 - 1];
      for ( nn = v9 - 1; nn > 0; --nn )
        p_Str1[nn] = p_Str1[nn - 1];
      *p_Str1 = v11;
    }
```

```
      break;
    case 5:
      v34 = v9;
      v26 = malloc(saturated_mul(v9, 4u));
      for ( i1 = 0; i1 < v9; ++i1 )
        v26[i1] = (Destination[i1 % v21] + 57) ^ p_Str1[i1];
      Source = sub_140001470((__int64)v26, v9);
      strcpy(p_Str1, Source);
      free(v26);
      free(Source);
      break;
    case 6:
      Size = saturated_mul(strlen(p_Str1), 4u);
      v27 = malloc(Size);
      sub_140001020((__int64)p_Str1, Destination, (__int64)v27);
      v4 = strlen(p_Str1);
      p_Str1_1 = sub_140001470((__int64)v27, v4);
      strcpy(p_Str1, p_Str1_1);
      free(v27);
      free(p_Str1_1);
      break;
    case 7:
      Size_1 = saturated_mul(strlen(p_Str1), 4u);
      v28 = malloc(Size_1);
      sub_140001250(p_Str1, Destination, v28);
      v6 = strlen(p_Str1);
      p_Str1_2 = sub_140001470((__int64)v28, v6);
      strcpy(p_Str1, p_Str1_2);
      free(v28);
      free(p_Str1_2);
      break;
  }
}
```

ps：函数sub_140001470（base64）

```
Size = 2 * a2;
if ( !is_mul_ok(2u, a2) )
  Size = -1;
v8 = malloc(Size);
v7 = 0;
v5 = -6;
v4 = 0;
for ( i = 0; i < a2; ++i )
{
  v7 = *(_DWORD *)(a1 + 4LL * i) + (v7 << 8);
  for ( v5 += 8; v5 >= 0; v5 -= 6 )
    v8[v4++] = Str[(v7 >> v5) & 0x3F];        // "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
}
if ( v5 > -6 )
  v8[v4++] = Str[(v7 << 8 >> (v5 + 8)) & 0x3F];// "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
while ( v4 % 4 )
  v8[v4++] = 61;
v8[v4] = 0;
return v8;
```

函数sub_140001020（仿rc4）

```
v8 = 0;
for ( n256 = 0; n256 < 256; ++n256 )
  v13[n256] = n256;
for ( n256_1 = 0; n256_1 < 256; ++n256_1 )
{
  v3 = Destination[n256_1 % strlen(Destination)] + v13[n256_1] + v8;
  v8 = (unsigned __int8)(BYTE4(v3) + v3) - BYTE4(v3);
  v11 = v13[n256_1];
  v13[n256_1] = v13[v8];
  v13[v8] = v11;
}
v9 = 0;
v7 = 0;
for ( i = 0; ; ++i )
{
  result = (unsigned int)*(char *)(p_Str1 + i);
  if ( !*(_BYTE *)(p_Str1 + i) )
    break;
  v7 = (v7 + 1) % 256;
  v9 = (v13[v7] + v9) % 256;
  v12 = v13[v7];
  v13[v7] = v13[v9];
  v13[v9] = v12;
  *(_DWORD *)(a3 + 4LL * i) = ((v13[(v13[v9] + v13[v7]) % 256] ^ *(char *)(p_Str1 + i)) + 57) % 256;
}
return result;
```

最阴的来了，开头调用了一次函数sub_140001F60，是一段base64换表，往后偏移了20位

（多看汇编少看伪码）

```
; __int64 sub_140001F60()
sub_140001F60 proc near
```

```
v5 = strlen(Str);                       // "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
v1 = 20 % v5;
if ( !(20 % v5) )
  return 0;
for ( i = 0; i < v1; ++i )
  v6[i] = Str[i + v5 - v1];             // "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
for ( j = v5 - 1; j >= v1; --j )
  Str[j] = Str[j - v1];                 // "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
for ( k = 0; k < v1; ++k )
  Str[k] = v6[k];                       // "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"
return 0;
```

反写脚本（长脚本写出来一堆问题，写吐了，最后ai修了好多，尤其是rc4和换表那里）

```
#include<bits/stdc++.h>
using namespace std;

int mod26(int x){
    x %= 26;
    if(x < 0) x += 26;
    return x;
}
```

```
int mod10(int x){
    x %= 10;
    if(x < 0) x += 10;
    return x;
}

string base64_decode(const string &in) {
    string table = "stuvwxyz0123456789+/ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqr";
    vector<int> T(256, -1);
    for (int i = 0; i < 64; i++) T[(unsigned char)table[i]] = i;

    string out;
    int val = 0, valb = -8;
    for (int i = 0; i < (int)in.size(); i++) {
        unsigned char c = (unsigned char)in[i];
        if (in[i] == '=') break;
        if (T[c] == -1) continue;
        val = (val << 6) + T[c];
        valb += 6;
        if (valb >= 0) {
            out.push_back(char((val >> valb) & 0xFF));
            valb -= 8;
        }
    }
    return out;
}
//一个base64解码的函数，后面会用到，这里要用换表后的base64

void rc4_init(int S[256], const char *Destination){
    int j = 0;
    int keylen = strlen(Destination);

    for (int i = 0; i < 256; ++i)
        S[i] = i;//定义并初始化s盒

    for (int i = 0; i < 256; ++i)
    {
        j = (j + S[i] + (unsigned char)Destination[i % keylen]) % 256;
        swap(S[i], S[j]);
    }
}
//用来解仿rc4的函数

void reverse_sub_1400015D0(char *p_Str1, const char *Destination, int a3)
{
    int i, j, k, ii, jj, kk, mm, nn, i1;
    int v9, v10, v11, v18, v21, v22, v25;
    char **Block;

    v9 = strlen(p_Str1);
    v21 = strlen(Destination);
    switch ( a3 )
    {
```

```
        case 1:
            for ( i = 0; i < v9; ++i )
            {
                v22 = (unsigned char)Destination[i % v21];
                if ( p_Str1[i] < 65 || p_Str1[i] > 90 )
                {
                    if ( p_Str1[i] < 97 || p_Str1[i] > 122 )
                    {
                        if ( p_Str1[i] >= 48 && p_Str1[i] <= 57 )
                            p_Str1[i] = mod10(p_Str1[i] - 48 - v22) + 48;
                    }
                    else
                    {
                        p_Str1[i] = mod26(p_Str1[i] - 97 - v22) + 97;
                    }
                }
                else
                {
                    p_Str1[i] = mod26(p_Str1[i] - 65 - v22) + 65;
                }
            }
            break;
            //+ v22全部改成- v22

        case 3:
        {
            v10 = (unsigned char)*Destination % 10 + 2;
            int rows = (v9 + v10 - 1) / v10;
            int rem = v9 % v10;
            int pos = 0;

            Block = (char **)malloc(v10 * sizeof(char *));
            for ( k = 0; k < v10; ++k )
            {
                Block[k] = (char *)malloc(rows + 1);
                memset(Block[k], 0, rows + 1);
            }

            for ( k = 0; k < v10; ++k )
            {
                int len;
                if ( rem == 0 || k < rem )
                    len = rows;
                else
                    len = rows - 1;

                for ( j = 0; j < len; ++j )
                    Block[k][j] = p_Str1[pos++];
            }

            v18 = 0;
            for ( ii = 0; ii < rows; ++ii )
            {
```

```
                for ( jj = 0; jj < v10; ++jj )
                {
                    if ( Block[jj][ii] && v18 < v9 )
                        p_Str1[v18++] = Block[jj][ii];
                }
            }

            p_Str1[v18] = 0;
            for ( kk = 0; kk < v10; ++kk )
                free(Block[kk]);
            free(Block);
            break;
        }
            //原来是按行读，按列输出，反写为按列读，按行输出

        case 4:
            v25 = (unsigned char)*Destination % 10 + 2;
            for ( mm = 0; mm < v25; ++mm )
            {
                v11 = p_Str1[0];
                for ( nn = 0; nn < v9 - 1; ++nn )
                    p_Str1[nn] = p_Str1[nn + 1];
                p_Str1[v9 - 1] = v11;
            }
            break;
            //指针循环右移,把循环里的顺序反过来 ,子循环改为逐增,nn-1改为nn+1

        case 5:
        {
            string tmp = base64_decode(p_Str1);   // 前面的解码函数
            for (i1 = 0; i1 < (int)tmp.size(); ++i1)
                p_Str1[i1] = (unsigned char)tmp[i1] ^ (((unsigned char)Destination[i1 % v21]
+ 57) & 0xFF);
            p_Str1[tmp.size()] = 0;
            break;
        }
            //sub_140001470是base64编码的函数。这一段是xor+base64

        case 6:
        {
            string tmp = base64_decode(p_Str1);//解base64
            int S[256];
            rc4_init(S, Destination);

            int x = 0, y = 0;
            for (i = 0; i < (int)tmp.size(); ++i)
            {
                x = (x + 1) % 256;
                y = (y + S[x]) % 256;
                swap(S[x], S[y]);

                int t = S[(S[x] + S[y]) % 256];
                p_Str1[i] = (unsigned char)((((unsigned char)tmp[i] - 57 + 256) % 256) ^ t);
```

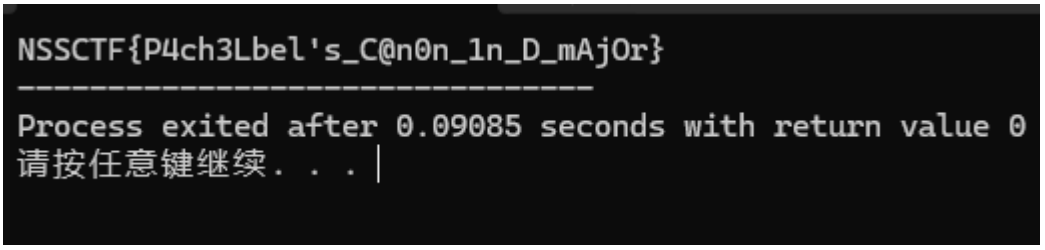
```
            }
            p_Str1[tmp.size()] = 0;
            break;
        }
            //原函数是rc4，异或+57，base64编码 （函数sub_140001020是仿rc4再异或再+57）
    }
}

int main() {
    char Destination[256] = "WgvDmssEvcY326bHo3nNro3vXvvfmgrz";
    char Str1[256] = "gX+Ri9PG=bt5=OOB6hscPQOL";
    char Destination_[256] = "T6bHgUPL2gXUd=xT=FNHtPzV";

    int a[] = {1,5,6,3,4,1,4,5};
    int v22[] = {0,1,2};
    int v11[] = {7,6,5};

    for (int n8_1 = 7; n8_1 >= 0; --n8_1)
    {
        for (int n2 = 2; n2 >= 0; --n2)
        {
            if ( n8_1 >= v22[n2] && v11[n2] >= 0 )
            {
                if ( n2 == 2 )
                {
                    reverse_sub_1400015D0(Destination_, Destination, a[v11[n2]]);
                }
                else if ( n2 == 1 )
                {
                    reverse_sub_1400015D0(Str1, Destination_, a[v11[n2]]);
                }
                else
                {
                    reverse_sub_1400015D0(Destination, Str1, a[v11[n2]]);
                }
                --v11[n2];
            }
        }
    }
    //仿制主函数写一个循环
    printf("%s%s%s", Destination, Str1, Destination_);
    return 0;
}
```

（本来想着分三次出结果，但是Str1,Destination，Destination_都会变，只能写一坨、仿制主函数套个循环了）



NSSCTF{P4ch3Lbel's_C@n0n_1n_D_mAjOr}

ps：另外标题的Canon应该指的是卡农的复调技法

Destination，Str1，Destination_好比三个声部，依次进入同一个循环（按时间依次进入同一段旋律）

此外{1,5,6,3,4,1,4,5}是和弦级数1-5-6m-3m-4-1-4-5，是D大调卡农里低音声部循环和弦D-A-Bm-F#m-G-D-G-A

.7032[VNCTF 2025]Fuko's starfish

一个exe一个dll

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  __int64 v3; // rax
  __int64 ?cerr@std@@3V?$basic_ostream@DU?$char_traits@D@std@@@1@A; // rax
  DWORD err; // [rsp+2Ch] [rbp-Ch]
  HMODULE hLibModule; // [rsp+30h] [rbp-8h]

  hLibModule = LoadLibraryW(L"starfish.dll");
  if ( hLibModule )
  {
    Sleep(0x7D0u);
    sub_140001170("\nhello ctfer\n");
    Sleep(0x7D0u);
    sub_140001170("I am Ibuki Fuko!\n");
    Sleep(0x7D0u);
    sub_140001170("If you want to get my starfish\n");
    Sleep(0x7D0u);
    sub_140001170("You need to complete my three games!\n");
    Sleep(0x7D0u);
    sub_140001170("Are you ready?\n");
    Sleep(0x7D0u);
    sub_140001170("Let's start!\n");
    Sleep(0x7D0u);
    system("cls");
    sub_140001490();
    sub_140001170("Goodbye\n");
    FreeLibrary(hLibModule);
    return 0;
  }
  else
  {
    err = GetLastError();
    v3 = sub_140001770(std::cerr, "Failed to load the DLL. Error code: ");
    ?cerr@std@@3V?$basic_ostream@DU?$char_traits@D@std@@@1@A = std::ostream::operator<<(v3, err);
    sub_140001770(?cerr@std@@3V?$basic_ostream@DU?$char_traits@D@std@@@1@A, "\n");
    return 1;
  }
}
```

进程序加载dll，过完剧情进函数sub_140001490

```
char sub_140001490()
{
  char result; // al
  __int64 v1; // [rsp+0h] [rbp-38h] BYREF
  int n2; // [rsp+2Ch] [rbp-Ch] BYREF
  __int64 v3; // [rsp+30h] [rbp-8h]

  while ( 1 )
  {
    sub_140001110();
    n2 = 0;
    result = sub_140001540("%d", (unsigned int)&n2);
    if ( n2 == 1 )
      break;
    if ( n2 == 2 )
      goto LABEL_6;
  }
  result = sub_140001270();
  if ( (result & 1) != 0 )
  {
    sub_140001170((__int64)"Congratulations, you are right!\n");
    sub_140001170((__int64)"Then, let's play Fuko's second game!\n");
    Sleep(0x7D0u);
    system("cls");
    result = play_snake();
  }
LABEL_6:
  if ( ((unsigned __int64)&v1 ^ v3) != _security_cookie )
    JUMPOUT(0x140001536LL);
  return result;
}
```

函数sub_140001270为第一关逻辑

```
char sub_140001270()
{
  unsigned int Seed; // eax
  char result; // al
  __int64 v2; // [rsp+0h] [rbp-38h] BYREF
  char v3; // [rsp+23h] [rbp-15h]
  int n10; // [rsp+24h] [rbp-14h]
  int v5; // [rsp+28h] [rbp-10h]
  int v6; // [rsp+2Ch] [rbp-Ch] BYREF
  __int64 v7; // [rsp+30h] [rbp-8h]

  Seed = sub_140001370(0);
  srand(Seed);
  v5 = rand() % 100 + 1;
  v6 = 0;
  for ( n10 = 0; n10 < 10; ++n10 )
  {
    sub_140001170((__int64)"please input a number :");
    sub_140001390("%d", (unsigned int)&v6);
    if ( v6 >= v5 )
    {
      if ( v6 <= v5 )
      {
        sub_140001170((__int64)"u win!\n");
        v3 = 1;
        goto LABEL_10;
      }
      sub_140001170((__int64)"too big!\n");
    }
    else
    {
      sub_140001170((__int64)"too small!\n");
    }
  }
  sub_140001170((__int64)"u lose!\n");
  v3 = 0;
LABEL_10:
  result = v3;
  if ( ((unsigned __int64)&v2 ^ v7) != _security_cookie )
    JUMPOUT(0x140001366LL);
  return result;
}
```

逻辑为10次以内猜到在1~100的一个随机数（要调试就在寄存器里抓，要玩也可以二分法7步以内过）

接下来看第二关逻辑play_snake，在dll里

是一个贪吃蛇游戏

```
while ( 1 )
{
  system("cls");
  puts("----------Snake-----------");
  puts("---------1.Play ----------");
  puts("--------------------------");
  puts("---------0.Esc  ----------");
  puts("--------------------------");
  v6 = 0;
  sub_1800030E0(v1, &v6);
  if ( !v6 )
    goto LABEL_12;
  qword_18000E604 = 0x200000001LL;
  n3 = 3;
  Seed = time64(0);
  srand(Seed);
```

通关后进入函数sub_1800025F0，然而这个函数是空的

```
__int64 sub_1800025F0()
{
  return 6;
}
```

而我们按流程走发现有第三关，开头的提示也是有三关

```
u win!
ok
its the last game!
plz input the final key:
```

搜索字符串找到u win，找到其汇编地址

```
lea     rcx, aUWin      ; "u win!"
call    puts
mov     ecx, 3E8h       ; dwMilliseconds
call    cs:Sleep
xorps   xmm0, xmm0
movaps  cs:xmmword_18000E310, xmm0
```

翻到底下找到sub_1800025F0的汇编地址

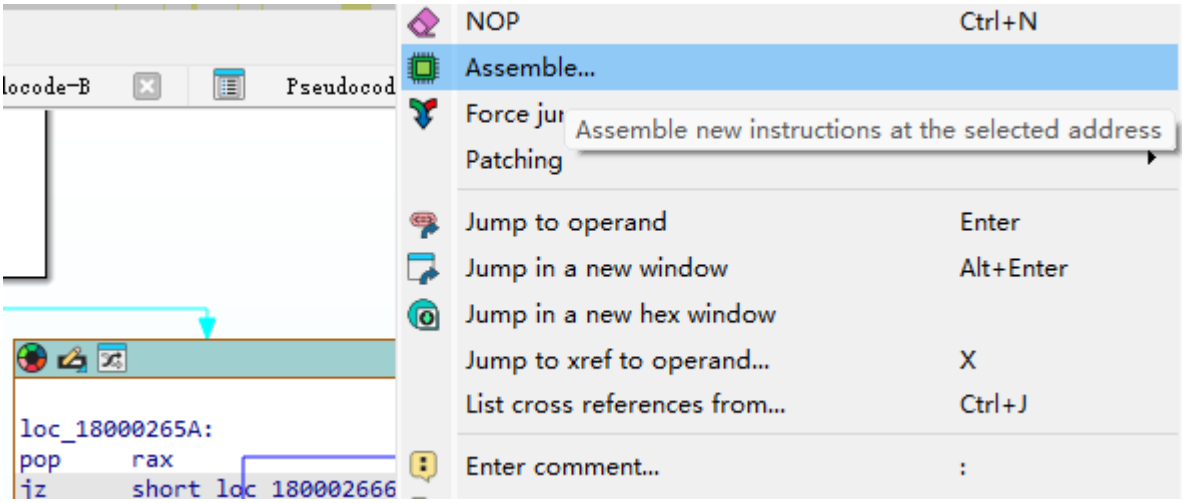
```
mov     cs:n3_0, 3
call    sub_1800025F0
```

互相跳转，经典花指令

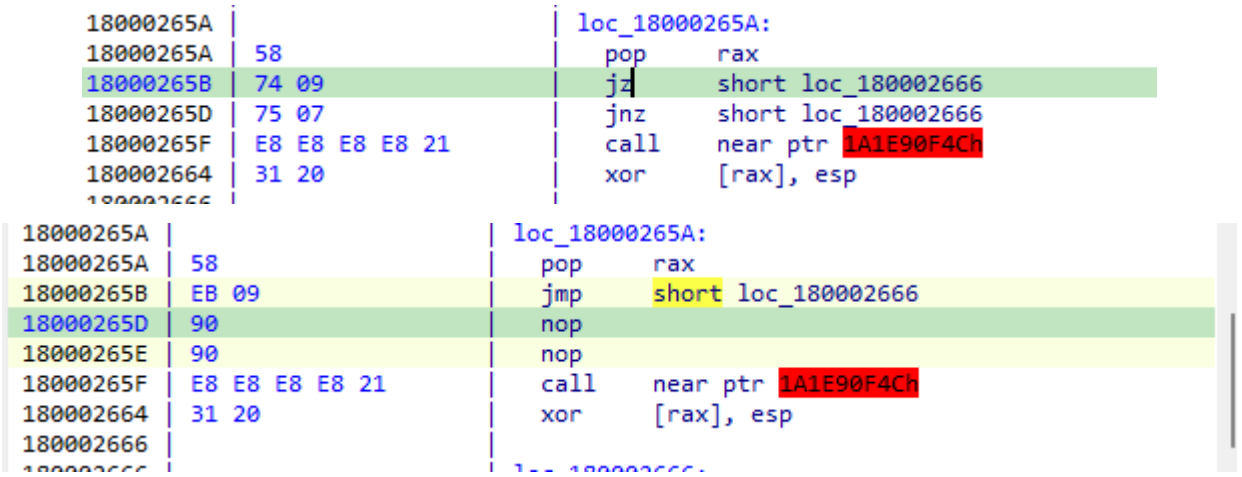
```
loc_18000265A:
pop     rax
jz      short loc_180002666
```

```
jnz     short loc_180002666
```

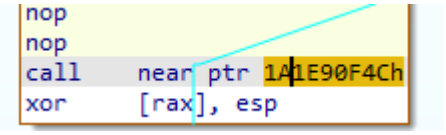
选中jz那一句



第一句改成jmp，第二句nop



爆红的地方按U取消定义



汇编里找到push rax改成jmp loc_180002666确定边界

最后Edit -> Functions -> Delete function删除函数，找到sub_1800025F0 开头，按P重建函数

正常看到sub_1800025F0的伪码

```
void __noreturn sub_1800025F0()
{
  size_t i; // rdi
  __int64 v1; // rcx
  __int64 v2; // rax
  const char *right; // rcx
  _BYTE n8[16]; // [rsp+20h] [rbp-60h] BYREF
  __int128 v5; // [rsp+30h] [rbp-50h]
  char Str[16]; // [rsp+90h] [rbp+10h] BYREF
  _BYTE v7[30]; // [rsp+A0h] [rbp+20h]
```

```
  __int128 v8; // [rsp+C0h] [rbp+40h]
  char v9; // [rsp+D0h] [rbp+50h]
  _BYTE v10[31]; // [rsp+D1h] [rbp+51h] BYREF
  char v11; // [rsp+F0h] [rbp+70h]
  __int128 v12; // [rsp+100h] [rbp+80h] BYREF
  unsigned __int8 v13; // [rsp+110h] [rbp+90h]

  *(_OWORD *)&v7[14] = *(__int128 *)((char *)&xmmword_18000A9C0 + 14);
  *(_OWORD *)v7 = xmmword_18000A9C0;
  *(_OWORD *)Str = xmmword_18000A9B0;
  for ( i = 0; strlen(Str) > i; ++i )
    Str[i] ^= 0x17u;
  puts(Str);
  sub_180002780(v1, n8);
  v11 = 0;
  v12 = 0;
  v13 = 0;
  *(_OWORD *)&v10[15] = v5;
  sub_180001650(&v10[15], &v12);
  v5 = 0;
  sub_180001650(n8, &v10[15]);
  v9 = v11;
  v8 = *(_OWORD *)&v10[15];
  *(_OWORD *)v10 = v12;
  *(_WORD *)&v10[16] = v13;
  v2 = 0;
  while ( *((_BYTE *)&v8 + 2 * v2) == byte_18000A890[2 * v2]
       && *((_BYTE *)&v8 + 2 * v2 + 1) == byte_18000A890[2 * v2 + 1] )
  {
    if ( ++v2 == 16 )
    {
      right = "right!";
      goto LABEL_10;
    }
  }
  right = "wrong";
LABEL_10:
  puts(right);
  Sleep(0xFA0u);
  exit(0);
}
```

v2 *2，要处理32个字节

```
while ( *((_BYTE *)&v8 + 2 * v2) == byte_18000A890[2 * v2]
     && *((_BYTE *)&v8 + 2 * v2 + 1) == byte_18000A890[2 * v2 + 1] )
{
  if ( ++v2 == 16 )
```

要校验的密文在byte_18000A890（正好32字节）

```
.rdata:000000018000A890 byte_18000A890  db 3Dh, 1, 1Ch, 19h, 0Bh, 0A0h, 90h, 81h, 5Fh, 67h, 27h
.rdata:000000018000A890                                         ; DATA XREF: sub_1800025F0+135↑o
.rdata:000000018000A89B                 db 31h, 0A8h, 9Ah, 0A4h, 74h, 97h, 36h, 21h, 67h, 0ABh
.rdata:000000018000A8A5                 db 2Eh, 0B4h, 0A0h, 94h, 18h, 0D3h, 7Dh, 93h, 0E6h, 46h
.rdata:000000018000A8AF                 db 0E7h
.rdata:000000018000A8B0 ; unsigned __int8 byte_18000A8B0[256]
```

校验逻辑是函数sub_180001650

反复*2和异或，有点像aes

```
for ( n4 = 0; n4 != 4; ++n4 )
{
  v72 = v187[n4 - 4];
  v73 = (2 * v72) ^ 0x1B;
  if ( v72 >= 0 )
    v73 = 2 * v72;
  v74 = v73;
  if ( n3 != 3 )
    v74 = 0;
  v75 = v189[n4];
```

反复在查表byte_18000A8B0

```
v31 = byte_18000A8B0[v16];
v32 = (unsigned __int8)v129 | (v30 << 8);
v33 = byte_18000A8B0[(unsigned __int8)v130];
v34 = byte_18000A8B0[(unsigned __int8)v137];
v149 = v32 | ((unsigned __int8)v130 << 16) | ((unsigned __int8)v137 <
v35 = *(_DWORD *)Str ^ (v34 | (byte_18000A8B0[(unsigned __int8)v129]
LODWORD(v150) = v35 ^ 0x1000000;
DWORD1(v150) = v147 ^ v35 ^ 0x1000000;
DWORD2(v150) = v148 ^ DWORD1(v150);
HIDWORD(v150) = v149 ^ v148 ^ DWORD1(v150);
v36 = v35
    ^ (byte_18000A8B0[((unsigned int)v149 ^ DWORD2(v150)) >> 24]
     | (byte_18000A8B0[(unsigned __int8)(v149 ^ v148 ^ v147 ^ v35)] <
     | (byte_18000A8B0[(unsigned __int16)(v32 ^ v148 ^ v147 ^ v35) >>
     | (byte_18000A8B0[(unsigned __int8)(((unsigned int)v149 ^ DWORD2
LODWORD(v151) = v36 ^ 0x3000000;
DWORD1(v151) = v36 ^ 0x3000000 ^ DWORD1(v150);
DWORD2(v151) = v148 ^ v36 ^ 0x3000000;
```

搜索了解到开头63 7C 77 7B F2 6B 6F C5 ...是aes的s盒特征

```
byte_18000A8B0  db 63h, 7Ch, 77h, 7Bh, 0F2h, 6Bh, 6Fh, 0C5h, 30h, 1, 67h
```

开头定义了16个key字节，对应aes128中16位key

```
v2 = byte_18000E1E0;
v3 = byte_18000E1F0;
v4 = byte_18000E1F2;
v5 = byte_18000E200;
v6 = byte_18000E204;
v7 = byte_18000E210;
v8 = byte_18000E950;
v9 = byte_18000E220;
LOBYTE(v138) = byte_18000E228;
v125 = byte_18000E230;
v126 = byte_18000E232;
v128 = byte_18000E240;
LOBYTE(v137) = byte_18000E244;
LOBYTE(v130) = byte_18000E960;
v127 = byte_18000E962;
LOBYTE(v129) = byte_18000E970;
```

找到对应字节，但是发现960，962，970没有数，初始值未知

```
.data:000000018000E1E0 byte_18000E1E0  db 11h
.data:000000018000E1E0
.data:000000018000E1E1                 align 10h
.data:000000018000E1F0 byte_18000E1F0  db 45h
.data:000000018000E1F0
.data:000000018000E1F1                 align 2
.data:000000018000E1F2 byte_18000E1F2  db 14h
.data:000000018000E1F2
.data:000000018000E1F3                 align 20h
.data:000000018000E200 byte_18000E200  db 19h
.data:000000018000E200
.data:000000018000E201                 align 4
.data:000000018000E204 byte_18000E204  db 19h
.data:000000018000E204
.data:000000018000E205                 align 10h
.data:000000018000E210 byte_18000E210  db 81h
.data:000000018000E210
.data:000000018000E211                 align 20h
.data:000000018000E220 byte_18000E220  db 11h
.data:000000018000E220
.data:000000018000E221                 align 8
.data:000000018000E228 byte_18000E228  db 45h
.data:000000018000E228
.data:000000018000E229                 align 10h
.data:000000018000E230 byte_18000E230  db 14h
.data:000000018000E230
.data:000000018000E231                 align 2
.data:000000018000E232 byte_18000E232  db 19h
.data:000000018000E232
.data:000000018000E233                 align 20h
.data:000000018000E240 byte_18000E240  db 19h
.data:000000018000E240
.data:000000018000E241                 align 4
.data:000000018000E244 byte_18000E244  db 81h

.data:000000018000E951                 align 20h
.data:000000018000E960 byte_18000E960  db ?
.data:000000018000E960
.data:000000018000E961                 align 2
.data:000000018000E962 byte_18000E962  db ?
.data:000000018000E962
.data:000000018000E963                 align 10h
.data:000000018000E970 byte_18000E970  db ?
.data:000000018000E970
```

没思路了，回到dllmain

搜索了解到if (fdwReason == 1)是dll被加载时的情况

这里创建了一个线程函数ThreadProc（其实经验丰富的话可以反应过来第一步就应该看ThreadProc）

```
if ( fdwReason == 1 )
{
  hHandle_1 = CreateThread(0, 0, (LPTHREAD_START_ROUTINE)ThreadProc, 0, 0, ThreadId);
  if ( hHandle_1 )
  {
    hHandle = hHandle_1;
    puts("loading...");
    WaitForSingleObject(hHandle, 0x7D0u);
    puts("Done.");
    CloseHandle(hHandle);
```

单看ThreadProc的伪码一头雾水

```
v1 = rand();
byte_18000E1E0 = v1 + v1 / 255;
v2 = rand();
byte_18000E1F0 = v2 + v2 / 255;
v3 = rand();
byte_18000E1F2 = v3 + v3 / 255;
v4 = rand();
byte_18000E200 = v4 + v4 / 255;
v5 = rand();
byte_18000E204 = v5 + v5 / 255;
v6 = rand();
byte_18000E210 = v6 + v6 / 255;
v7 = rand();
byte_18000E950 = v7 + v7 / 255;
v8 = rand();
byte_18000E220 = v8 + v8 / 255;
v9 = rand();
byte_18000E228 = v9 + v9 / 255;
v10 = rand();
byte_18000E230 = v10 + v10 / 255;
v11 = rand();
byte_18000E232 = v11 + v11 / 255;
v12 = rand();
byte_18000E240 = v12 + v12 / 255;
v13 = rand();
byte_18000E244 = v13 + v13 / 255;
v14 = rand();
byte_18000E960 = v14 + v14 / 255;
v15 = rand();
byte_18000E962 = v15 + v15 / 255;
v16 = rand();
byte_18000E970 = v16 + v16 / 255;
return 6;
```

他的明文是固定的，那么key肯定是固定的，rand()本身就是伪随机，需要看设立的状态

依旧多看汇编少看伪码

不知为何这一段伪码没写出来

```
loc_1800033F3:
pop     rax
mov     ecx, 1BF52h     ; Seed
call    cs:srand
call    rsi ; rand
```

这一段翻译过来就是srand(114514) （1BF52h十六进制转十进制为114514）

srand设立了一个状态，种子为114514，相当于把rand()随机数的调用给固定了

再看下面，结合伪码中的v+v/255

cl是取字节的最后8位，就相当于x%255

```
imul    rcx, rax, 0FFFFFFFF80808081h
shr     rcx, 20h
add     ecx, eax
mov     edx, ecx
shr     edx, 1Fh
shr     ecx, 7
add     ecx, edx
add     ecx, eax
mov     cs:byte_18000E1E0, cl
call    rsi ; rand
```

再回到之前的校验函数sub_180001650

```
v124 = v2 ^ 0x17;
v28 = v3 ^ 0x17;
v29 = v4 ^ 0x17;
v27 = v5 ^ 0x17;
v21 = v7 ^ 0x17;
v22 = v8 ^ 0x17;
v24 = v9 ^ 0x17;
v17 = v138 ^ 0x17;
v13 = v125 ^ 0x17;
v19 = v6 ^ 0x17;
v18 = v126 ^ 0x17;
v20 = v128 ^ 0x17;
LOBYTE(v137) = v137 ^ 0x17;
LOBYTE(v130) = v130 ^ 0x17;
v16 = v127 ^ 0x17;
LOBYTE(v129) = v129 ^ 0x17;
v23 = v124;
v25 = v28;
v26 = v29;
```

总结逻辑：srand(114514)种子下取16次rand()-->%255-->^0x17

求key脚本

```
int main(){
  int key[16];
  srand(114514);
  for(int i=0;i<16;i++){
   key[i] = rand()%255;
   key[i] ^= 0x17;
  }

  for(int i=0;i<16;i++){
    printf("%x",key[i]);
  }

    return 0;
}
```

9e5fdeb683175b6b13b84891eb78d2

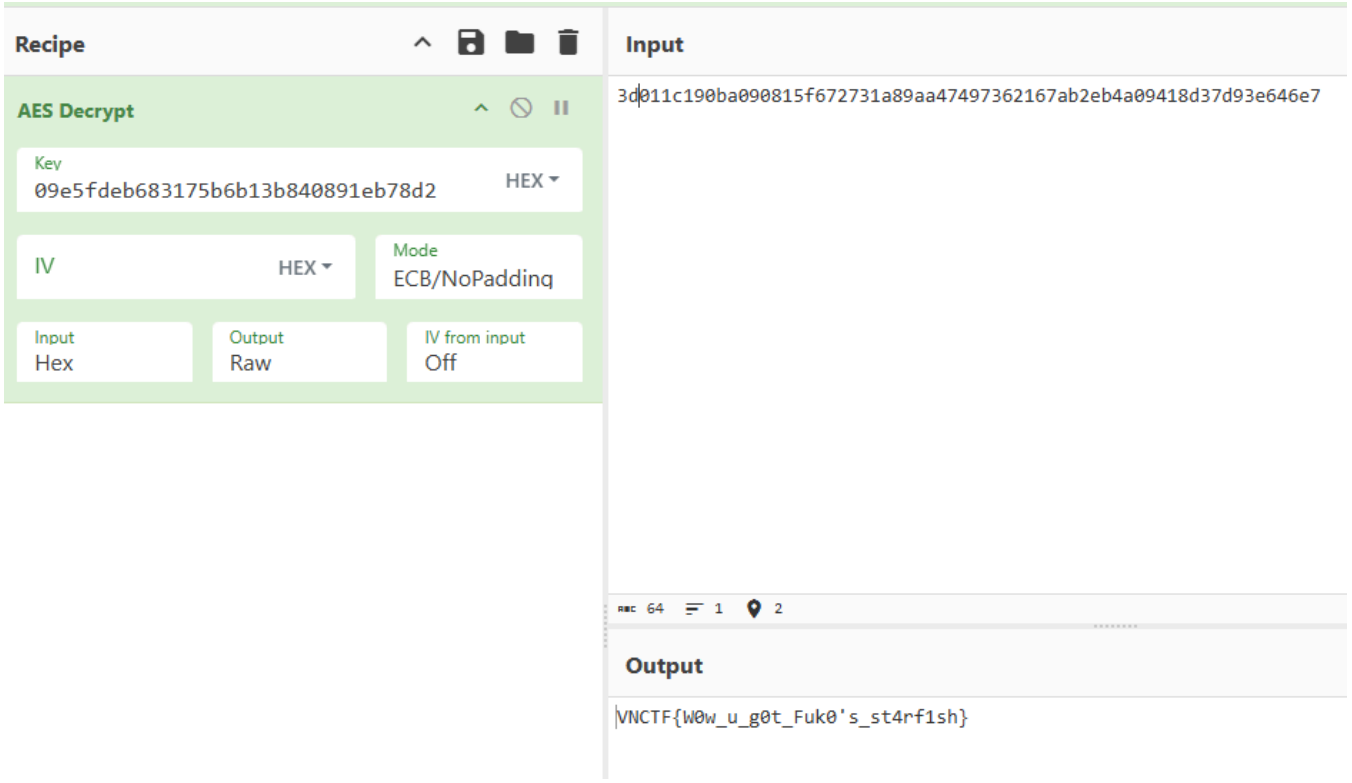
```
9e5fdeb683175b6b13b84891eb78d2
--------------------------------
Process exited after 0.08279 seconds with return value 0
请按任意键继续. . .
```

之前分析过是AES128

要校验的密文在byte_18000A890

```
.rdata:000000018000A890 byte_18000A890  db 3Dh, 1, 1Ch, 19h, 0Bh, 0A0h, 90h, 81h, 5Fh, 67h, 27h
.rdata:000000018000A890                                         ; DATA XREF: sub_1800025F0+135↑o
.rdata:000000018000A89B                 db 31h, 0A8h, 9Ah, 0A4h, 74h, 97h, 36h, 21h, 67h, 0ABh
.rdata:000000018000A8A5                 db 2Eh, 0B4h, 0A0h, 94h, 18h, 0D3h, 7Dh, 93h, 0E6h, 46h
.rdata:000000018000A8AF                 db 0E7h
```

填入key和密文，模式选ECB/NoPadding（没有iv选ECB，key是16字节的倍数的话就不用改变其填充）



VNCTF{W0w_u_g0t_Fuk0's_st4rf1sh}

.7309[SWPUCTF 2025 秋季新生赛]ez_tea

从start开始一路跳到sub_140013D60

一个key：1145141919810!!!

与目标密文byte_140020000

```
__int64 sub_140013D60()
{
  char *v0; // rdi
  __int64 n590; // rcx
  _BYTE v3[32]; // [rsp+0h] [rbp-20h] BYREF
  char v4; // [rsp+20h] [rbp+0h] BYREF
  _QWORD v5[19]; // [rsp+30h] [rbp+10h] BYREF
  __int64 v6; // [rsp+C8h] [rbp+A8h]
  _QWORD v7[259]; // [rsp+F0h] [rbp+D0h] BYREF
  char Destination[44]; // [rsp+908h] [rbp+8E8h] BYREF
  int i; // [rsp+934h] [rbp+914h]

  v0 = &v4;
  for ( n590 = 590; n590; --n590 )
  {
    *(_DWORD *)v0 = -858993460;
    v0 += 4;
  }
  sub_1400113F2(&unk_140026102);
  if ( IsDebuggerPresent() )
  {
    sub_140011285();
    exit(0);
  }
  sub_1400110A5(&unk_14001D34C, v5);
  v6 = sub_14001143D(v5);
  j_strcpy(Destination, Source);                // "1145141919810!!!"
  sub_140011483(v5, v7, v6, Destination);
  for ( i = 0; i < (unsigned __int64)(v6 + 4); ++i )
  {
    if ( *((unsigned __int8 *)v7 + i) != (unsigned __int8)byte_140020000[i] )
    {
      sub_1400111DB(&unk_14001D3C8);
      exit(0);
    }
  }
  sub_1400111DB(&unk_14001D3D8);
  sub_14001137A(v3, &unk_14001E110);
  return 0;
}
```

看校验函数sub_140011483再一路跳到sub_140014500

这一段已经有TEA的特征了

> v11 = (DWORD *)(i + a1);
> v12 = (DWORD *)(a1 + i + 4);

这段是把每8个字节拆成两个4字节再进行处理

```
__int64 __fastcall sub_140014500(__int64 a1, __int64 a2, unsigned __int64 a3, const void *a4)
{
  char *v4; // rdi
  __int64 n38; // rcx
  _BYTE v7[32]; // [rsp+0h] [rbp-20h] BYREF
  char v8; // [rsp+20h] [rbp+0h] BYREF
  _BYTE v9[48]; // [rsp+28h] [rbp+8h] BYREF
  unsigned __int64 i; // [rsp+58h] [rbp+38h]
  _DWORD *v11; // [rsp+78h] [rbp+58h]
  _DWORD *v12; // [rsp+98h] [rbp+78h]

  v4 = &v8;
  for ( n38 = 38; n38; --n38 )
  {
    *(_DWORD *)v4 = -858993460;
    v4 += 4;
  }
  sub_1400113F2(&unk_140026102);
  j_memcpy(v9, a4, 0x10u);
  for ( i = 0; i < a3; i += 8LL )
  {
    v11 = (_DWORD *)(i + a1);
    v12 = (_DWORD *)(a1 + i + 4);
    sub_14001132F(i + a1, v12, v9);
    *(_DWORD *)(i + a2) = *v11;
    *(_DWORD *)(i + a2 + 4) = *v12;
  }
  return sub_14001137A(v7, &unk_14001CF60);
}
```

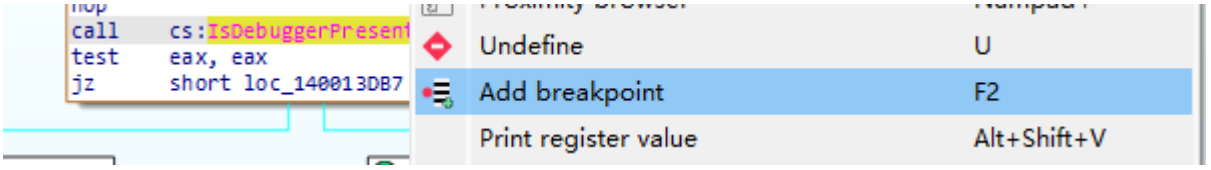
从sub_14001132F再一路跳到sub_1400138B0

```
__int64 __fastcall sub_1400138B0(_DWORD *a1, _DWORD *a2, __int64 a3)
{
  __int64 result; // rax
  unsigned int v4; // [rsp+24h] [rbp+4h]
  int n32; // [rsp+64h] [rbp+44h]

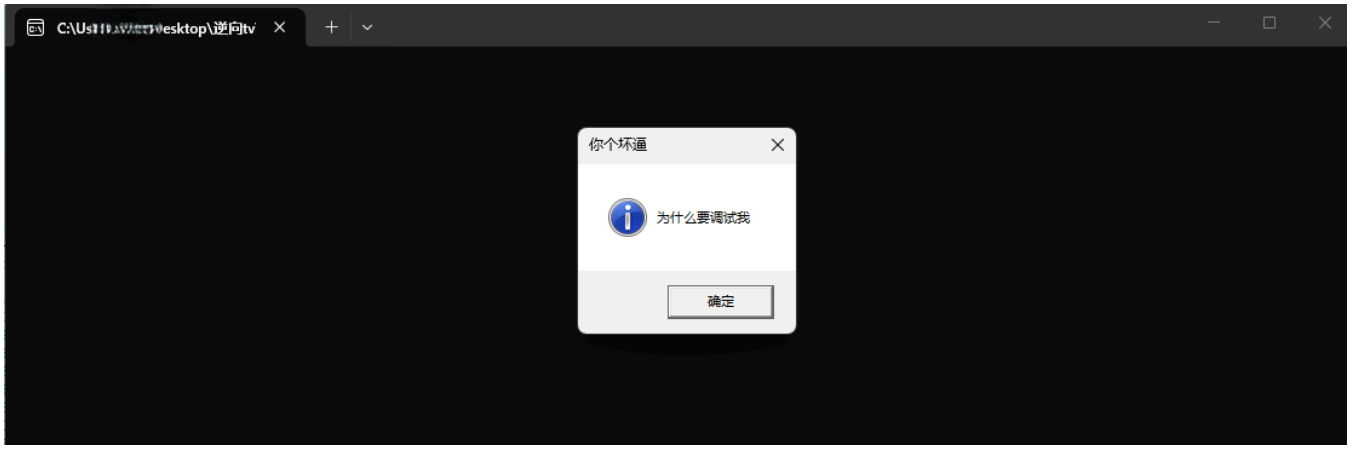
  result = sub_1400113F2(&unk_140026102);
  v4 = 0;
  for ( n32 = 0; n32 < 32; ++n32 )
  {
    v4 += 218105633;
    *a1 += (*a2 >> 5) ^ (*(_DWORD *)(a3 + 4LL * ((v4 >> 11) & 3)) + 16 * *a2);
    *a2 += (*a1 >> 5) ^ (*(_DWORD *)(a3 + 4LL * (((unsigned __int8)(v4 >> 11) ^ 1) & 3)) + 16 * *a1);
    result = (unsigned int)(n32 + 1);
  }
  return result;
}
```

额到这里我以为已经出来了，反写完脚本发现是乱码

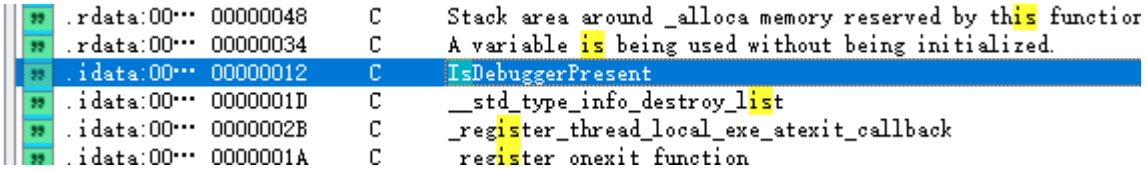
在call　cs:IsDebuggerPresent下断点（IsDebuggerPresent是判断是否在进行反调试的函数）



直接运行被拦截了



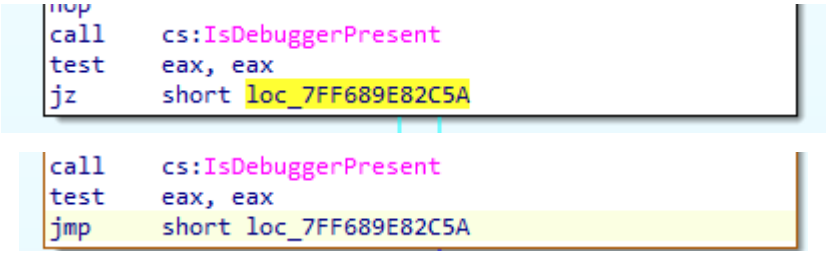
搜字符串发现另一处调用的地点：sub_7FF7F8152B90（运行时地址会变，直接看后面四位左右是一样的）



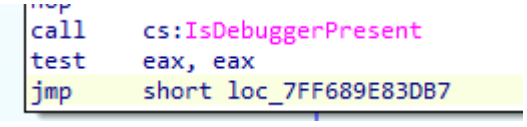

```
.idata:00007FF7F8164000 ; BOOL (__stdcall *IsDebuggerPresent)()
.idata:00007FF7F8164000                 extrn IsDebuggerPresent:qword
.idata:00007FF7F8164000                                         ; CODE XREF: sub_7FF7F8152B90+B7↑p
.idata:00007FF7F8164000                                         ; sub_7FF7F8153D60+3E↑p ...
.idata:00007FF7F8164008 ; HANDLE (__stdcall *GetCurrentProcess)()
```

但下了断点发现停不到位置，改用patch

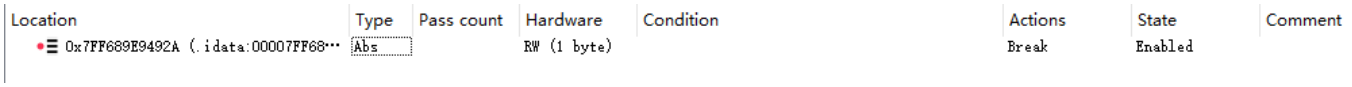
sub_7FF7F8152B90把jz改成jmp



sub_7FF689E83D60同理



ctrl alt b打开断点窗口，把断点都删掉



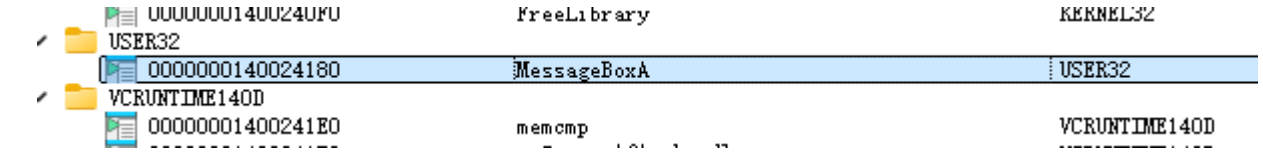
结果还是一如既往的弹窗拦截，

红温了把调用了IsDebuggerPresent的全部nop掉还是一样

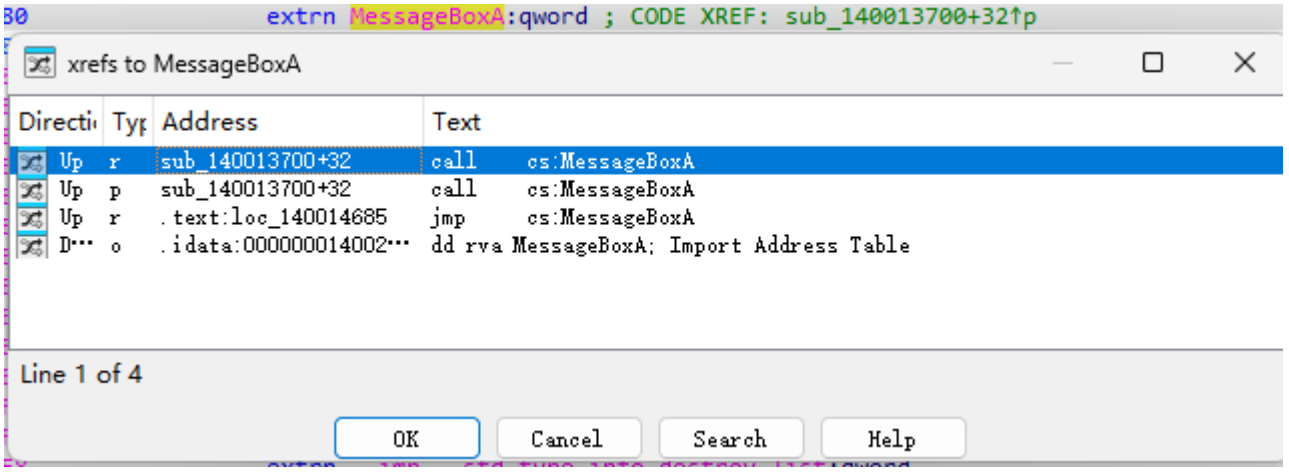
那应该不只有IsDebuggerPresent这一条反调试

（后来发现是在sub_140014500里有变化key）

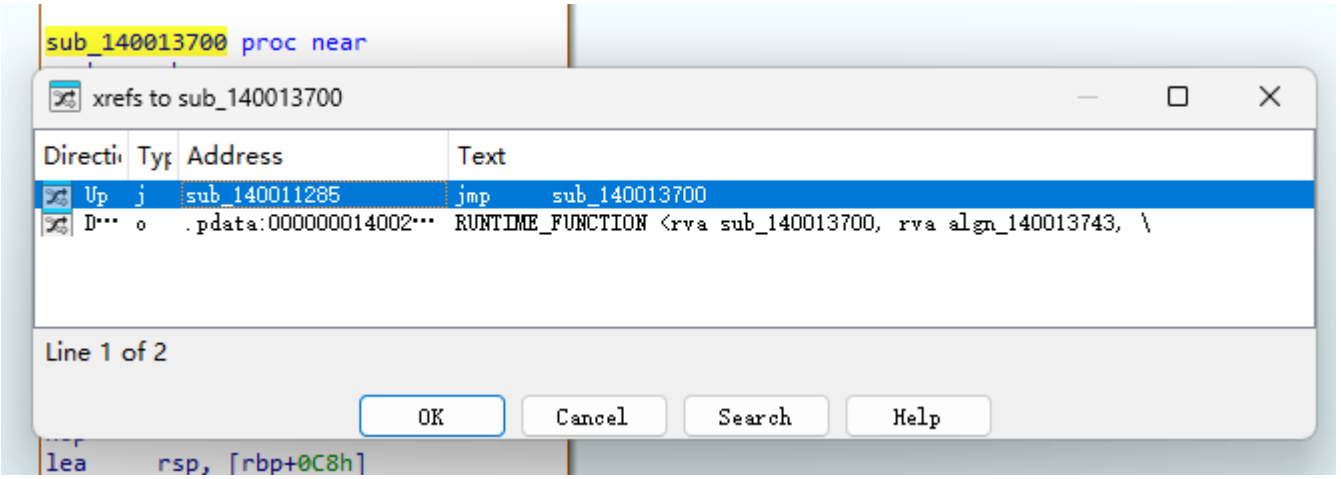
换个思路在Imports里面找到弹窗函数



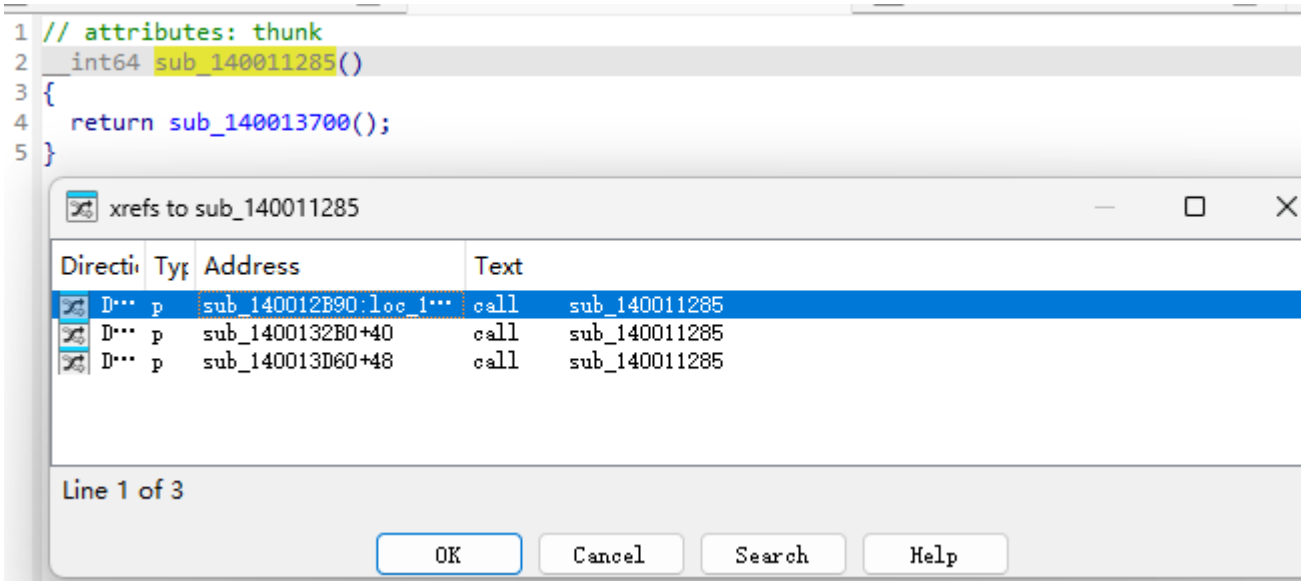
查看调用



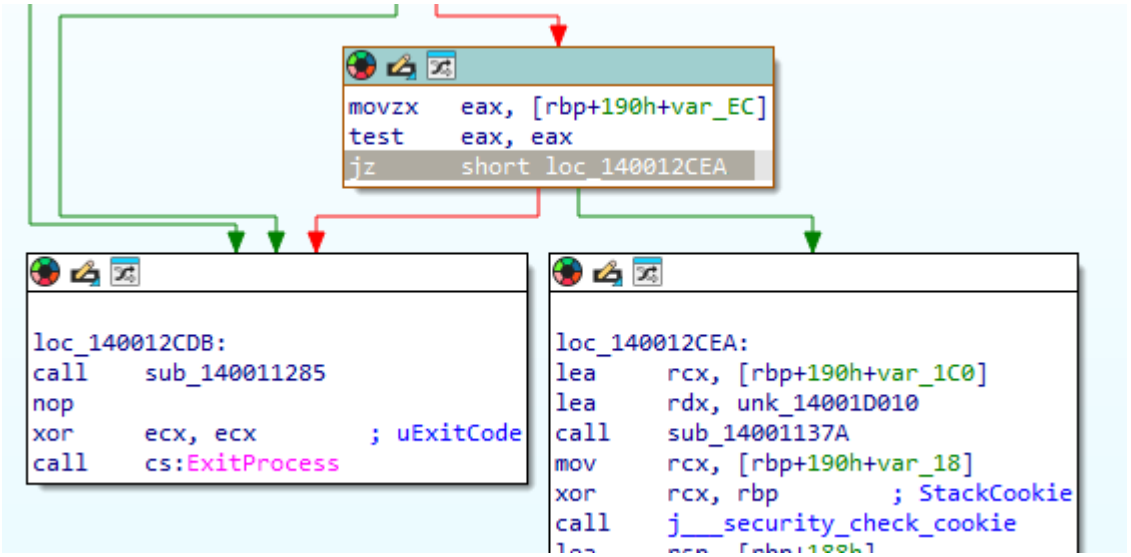
一路追到sub_140011285

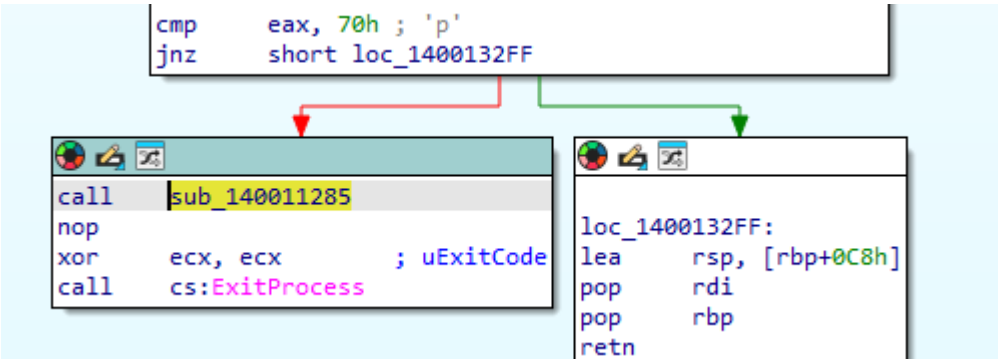


跳板函数，找调用

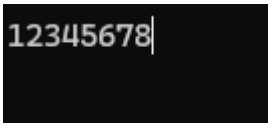


把jz改成jmp（前两个要改，最后一个原本就是jmp）





之后运行，发现弹窗没了，可以正常输入数字了

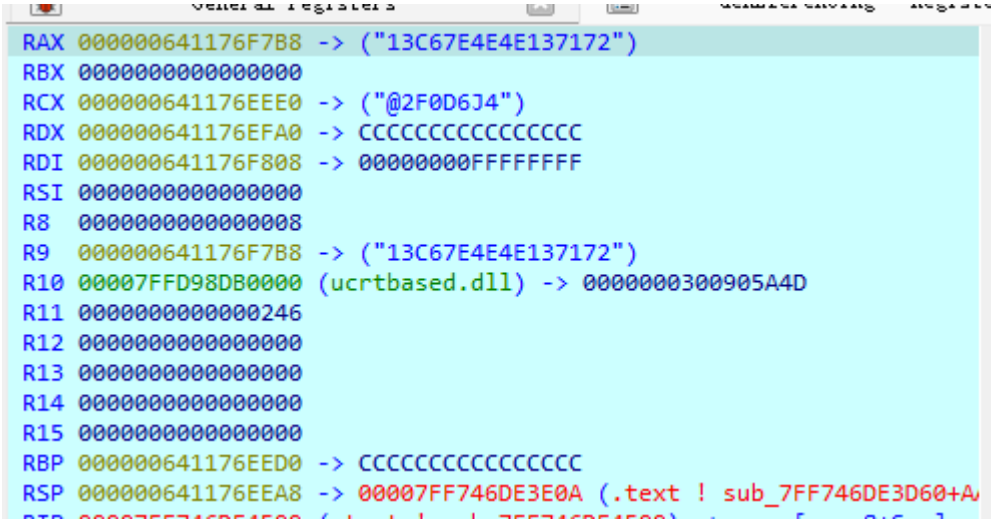


sub_140014500里下断点



运行，随便输一串字符，回车

寄存器里的key不是前面的114514，果然被改了



key：13C67E4E4E137172

接下来针对魔改TEA去写脚本

（自己写的脑袋尖尖的，最后ai改的基本上没有手写的成分了。。）

```
#define DELTA  0x0D000721u
#define ROUNDS 32

uint32_t key[4];

/*
 * 从小端字节序读取 4 字节整数。
 * 逆向里 _DWORD * 强转通常就是按 4 字节小端读取。
 */
uint32_t read_u32_le(const unsigned char *p) {
    return ((uint32_t)p[0])
         | ((uint32_t)p[1] << 8)
         | ((uint32_t)p[2] << 16)
         | ((uint32_t)p[3] << 24);
}

/*
 * 把 32 位整数按小端字节序写回内存。
 */
void write_u32_le(unsigned char *p, uint32_t x) {
    p[0] = x & 0xff;
    p[1] = (x >> 8) & 0xff;
    p[2] = (x >> 16) & 0xff;
    p[3] = (x >> 24) & 0xff;
}

/*
 * sub_1400138B0 的逆过程。
 *
 * 原加密逻辑大概是：
 *   sum += DELTA;
```

```
 *   v0 += (v1 >> 5) ^ (key[(sum >> 11) & 3] + (v1 << 4));
 *   v1 += (v0 >> 5) ^ (key[((sum >> 11) ^ 1) & 3] + (v0 << 4));
 *
 * 逆向解密时：
 *   1. sum 从 DELTA * 32 开始
 *   2. 先逆 v1
 *   3. 再逆 v0
 *   4. sum -= DELTA
 */
void decrypt_block(unsigned char *block) {
    uint32_t v0 = read_u32_le(block);
    uint32_t v1 = read_u32_le(block + 4);
    uint32_t sum = DELTA * ROUNDS;

    for (int i = 0; i < ROUNDS; i++) {
        uint32_t idx2 = (((sum >> 11) & 0xff) ^ 1) & 3;
        v1 -= (v0 >> 5) ^ (key[idx2] + (v0 << 4));

        uint32_t idx1 = (sum >> 11) & 3;
        v0 -= (v1 >> 5) ^ (key[idx1] + (v1 << 4));

        sum -= DELTA;
    }

    write_u32_le(block, v0);
    write_u32_le(block + 4, v1);
}

/*
 * sub_140013320 的逆过程。
 *
 * 原程序在加密前会先改输入。
 * 偶数长度时：
 *   s[i]     = (s[i] + 1) ^ 0x72;
 *   s[i + 1] = (s[i + 1] - 2) ^ 0x02;
 *
 * 所以逆过程是：
 *   s[i]     = (s[i] ^ 0x72) - 1;
 *   s[i + 1] = (s[i + 1] ^ 0x02) + 2;
 */
void reverse_input_mutation(unsigned char *buf, int len) {
    for (int i = 0; i < len; i += 2) {
        buf[i] = ((buf[i] ^ 0x72) - 1) & 0xff;
        buf[i + 1] = ((buf[i + 1] ^ 0x02) + 2) & 0xff;
    }
}

int main() {
    /*
     * byte_140020000，程序里拿加密结果和它比较。
     */
    unsigned char enc[] = {
        0xE8,0x1C,0xEB,0xDD,0x91,0x66,0x69,0x46,
```

```
        0xFD,0xF5,0xBA,0x47,0xED,0x25,0xB6,0x73,
        0x99,0xC4,0x3D,0xFD,0xFB,0x7E,0x9E,0x01,
        0x40,0x73,0xF8,0x9B,0xB4,0x48,0x8B,0xF2,
        0x5A,0xBE,0x7F,0xC0,0x6E,0xBE,0x40,0xD2,
        0x6B,0xFF,0x91,0xB0,0xF8,0x0B,0x0F,0x7D
    };

    /*
     * 动调在 sub_140014500 入口看到的真实 key。
     * 静态的 "1145141919810!!!" 是误导。
     */
    unsigned char key_bytes[] = "13C67E4E4E137172";

    /*
     * 原程序把 key 当作 4 个 DWORD 使用。
     */
    for (int i = 0; i < 4; i++) {
        key[i] = read_u32_le(key_bytes + i * 4);
    }

    int enc_len = sizeof(enc);

    /*
     * 先对全部密文按 8 字节分组解密。
     */
    for (int i = 0; i < enc_len; i += 8) {
        decrypt_block(enc + i);
    }

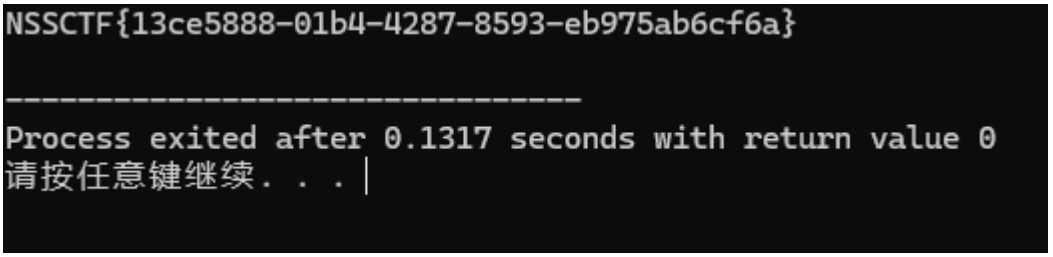
    /*
     * 程序比较的是 v6 + 4 字节，所以密文末尾可能带调试填充/多余数据。
     * 这里枚举偶数长度，找到以 } 结尾的可打印结果。
     */
    for (int n = 2; n <= enc_len; n += 2) {
        unsigned char tmp[64];

        memcpy(tmp, enc, n);
        tmp[n] = 0;

        reverse_input_mutation(tmp, n);

        if (tmp[n - 1] == '}') {
            printf("%s\n", tmp);
        }
    }

    return 0;
}
```



NSSCTF{13ce5888-01b4-4287-8593-eb975ab6cf6a}

水题

.2145[NSSCTF 2022 Spring Recruit]easy C

a进行逐位异或与b校验

```c
#include <stdio.h>
#include <string.h>

int main(){
    char a[]="wwwwwww";
    char b[]="d`vxbQd";

    //try to find out the flag
    printf("please input flag:");
    scanf(" %s",&a);

    if(strlen(a)!=7){
        printf("NoNoNo\n");
        system("pause");
        return 0;
    }

    for(int i=0;i<7;i++){
        a[i]++;
        a[i]=a[i]^2;
    }

    if(!strcmp(a,b)){
        printf("good!\n");
        system("pause");
        return 0;
    }

    printf("NoNoNo\n");
    system("pause");
    return 0;
    //flag 记得包上 NSSCTF{} 再提交!!!
}
```

反写脚本，先与2异或再减

```
int main(){
char a[] = "d`vxbQd";
for(int i=0;i<7;i++)
{
        a[i]=a[i]^2;
        a[i]--;
}
printf("%s",a);
}
```

NSSCTF{easy_re}



.2903[HNCTF 2022 Week1]贝斯是什么乐器啊?

拖进IDA看伪码

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  char Str2[160]; // [rsp+20h] [rbp-60h] BYREF
  char Str[124]; // [rsp+C0h] [rbp+40h] BYREF
  int i; // [rsp+13Ch] [rbp+BCh]

  _main();
  puts("please input your flag:");
  scanf("%s", Str);
  for ( i = 0; i < strlen(Str); ++i )
    Str[i] -= i;
  base64_encode(Str2, Str);
  if ( !strcmp(
          enc,                                  // "TlJRQFBBdTs4alsrKFI6MjgwNi5p"
          Str2) )
    printf("yes!");
  else
    printf("error");
  return 0;
}
```

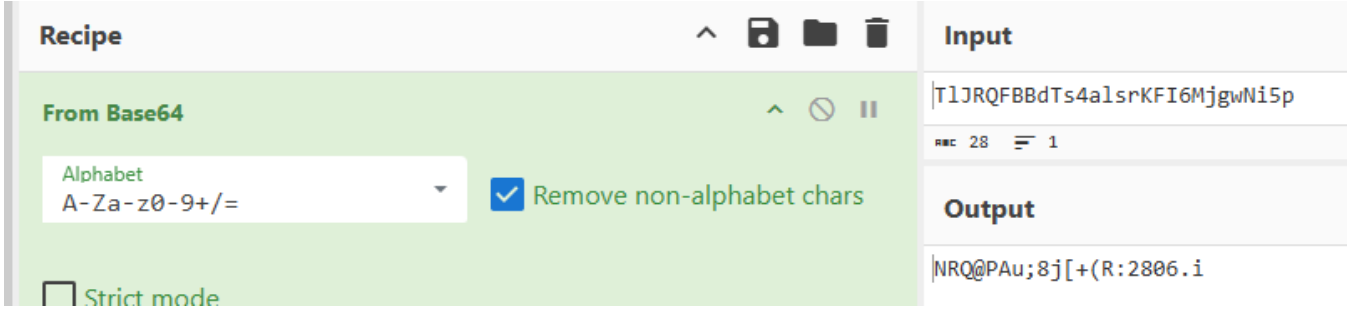
定义了两个字符，str2存字符串，str存用户输入的数据

重点关注for那句循环语句：逻辑是第i位减i

例如输入abcdefg就会变成aaaaaaa

然后把输出的str进行base64编码放进str2，与enc"TlJRQFBBdTs4alsrKFI6MjgwNi5p"进行校验

我们反着来，先对enc进行base64解密

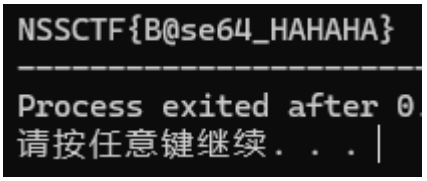


再写脚本，反过来第i位加i

```
int main(){
   char a[]="NRQ@PAu;8j[+(R:2806.i";

   for(int i=0;i<strlen(a);i++)
   a[i] += i;

   printf("%s",a);
}
```



NSSCTF{B@se64_HAHAHA}

##